

Q3. Inventory Management System — Test Case Pseudocode

Module Under Test: Update Stock | **Modules:** Add Item, Update Stock, Remove Item

Overview

The pseudocode below defines a structured approach to generating and executing test cases for the **Update Stock** functionality of an Inventory Management System. Test cases cover three key areas: (1) item availability validation, (2) quantity limit checks, and (3) expected system responses for both valid and invalid inputs.

1. Constants, Data Structures & Setup

```
// ■■■■ CONSTANTS ■■■■
CONST MAX_STOCK_LIMIT    = 10000    // Upper boundary for stock quantity
CONST MIN_STOCK_LIMIT    = 0        // Lower boundary (cannot be negative)
CONST VALID_ITEM_ID      = 'ITM001'
CONST INVALID_ITEM_ID    = 'UNKNOWN999'

// ■■■■ INVENTORY DATA STORE (simulated) ■■■■
INVENTORY = {
    'ITM001': { name: 'Laptop',    stock: 50,    active: TRUE },
    'ITM002': { name: 'Mouse',     stock: 200,   active: TRUE },
    'ITM003': { name: 'Keyboard',  stock: 0,     active: TRUE },
    'ITM004': { name: 'Monitor',   stock: 9990, active: FALSE }
}

// ■■■■ TEST RESULT TRACKER ■■■■
testResults = []    // stores { testID, description, status, actual }
```

2. Helper Functions

```
// Logs pass/fail result for each test case
FUNCTION logResult(testID, description, expected, actual):
    IF actual == expected THEN
        status = 'PASS'
    ELSE
        status = 'FAIL'
    END IF
    APPEND { testID, description, status, actual } TO testResults
    PRINT testID + ' [' + status + ']' + description
END FUNCTION

// Core system function being tested
FUNCTION updateStock(itemID, quantity):
    IF itemID NOT IN INVENTORY THEN
        RETURN 'ERROR: Item not found'
    END IF
    IF INVENTORY[itemID].active == FALSE THEN
```

```

        RETURN 'ERROR: Item is inactive'
    END IF
    IF quantity < MIN_STOCK_LIMIT OR quantity > MAX_STOCK_LIMIT THEN
        RETURN 'ERROR: Quantity out of allowed range'
    END IF
    IF quantity IS NOT INTEGER THEN
        RETURN 'ERROR: Invalid quantity type'
    END IF
    INVENTORY[itemID].stock = quantity
    RETURN 'SUCCESS: Stock updated to ' + quantity
END FUNCTION

```

3. Test Case Generation & Execution

3a. Item Availability Validation Tests

```

FUNCTION testItemAvailability():
    PRINT '--- Item Availability Tests ---'
    // TC-01: Valid active item
    result = updateStock('ITM001', 60)
    logResult('TC-01', 'Update stock for valid active item',
        'SUCCESS: Stock updated to 60', result)
    // TC-02: Item ID does not exist
    result = updateStock('UNKNOWN999', 10)
    logResult('TC-02', 'Update stock for non-existent item ID',
        'ERROR: Item not found', result)
    // TC-03: Inactive/discontinued item
    result = updateStock('ITM004', 50)
    logResult('TC-03', 'Update stock for inactive item',
        'ERROR: Item is inactive', result)
    // TC-04: Empty item ID string
    result = updateStock('', 10)
    logResult('TC-04', 'Update stock with empty item ID',
        'ERROR: Item not found', result)
    // TC-05: NULL item ID
    result = updateStock(NULL, 10)
    logResult('TC-05', 'Update stock with NULL item ID',
        'ERROR: Item not found', result)
END FUNCTION

```

3b. Quantity Limit Validation Tests

```

FUNCTION testQuantityLimits():
    PRINT '--- Quantity Limit Tests ---'
    // TC-06: Exact minimum boundary (0)

```

```

result = updateStock('ITM001', 0)
logResult('TC-06', 'Set stock to minimum boundary value (0)',
          'SUCCESS: Stock updated to 0', result)
// TC-07: Just below minimum (negative)
result = updateStock('ITM001', -1)
logResult('TC-07', 'Set stock to -1 (below minimum)',
          'ERROR: Quantity out of allowed range', result)
// TC-08: Exact maximum boundary (10000)
result = updateStock('ITM001', 10000)
logResult('TC-08', 'Set stock to maximum boundary value (10000)',
          'SUCCESS: Stock updated to 10000', result)
// TC-09: Just above maximum (10001)
result = updateStock('ITM001', 10001)
logResult('TC-09', 'Set stock to 10001 (above maximum)',
          'ERROR: Quantity out of allowed range', result)
// TC-10: Typical mid-range value
result = updateStock('ITM001', 500)
logResult('TC-10', 'Set stock to normal mid-range value (500)',
          'SUCCESS: Stock updated to 500', result)
// TC-11: Decimal/float quantity
result = updateStock('ITM001', 10.5)
logResult('TC-11', 'Set stock to decimal value (10.5)',
          'ERROR: Invalid quantity type', result)
// TC-12: String instead of integer
result = updateStock('ITM001', 'fifty')
logResult('TC-12', 'Set stock to string value',
          'ERROR: Invalid quantity type', result)
END FUNCTION

```

3c. System Response Validation Tests

```

FUNCTION testSystemResponse():
    PRINT '--- System Response Tests ---'
    // TC-13: Verify stock value is persisted after update
    updateStock('ITM002', 150)
    actual = INVENTORY['ITM002'].stock
    logResult('TC-13', 'Stock value persists in system after update',
              150, actual)
    // TC-14: Verify failed update does NOT change stock
    originalStock = INVENTORY['ITM001'].stock
    updateStock('ITM001', -5) // invalid – should fail
    actual = INVENTORY['ITM001'].stock
    logResult('TC-14', 'Failed update leaves stock unchanged',
              originalStock, actual)

```

```

// TC-15: Sequential updates – only last valid update retained
updateStock('ITM001', 100)
updateStock('ITM001', 200)
actual = INVENTORY['ITM001'].stock
logResult('TC-15', 'Sequential updates: final value is last valid update',
          200, actual)

// TC-16: Concurrent update simulation (bulk check)
FOR each itemID IN ['ITM001','ITM002','ITM003'] DO
    result = updateStock(itemID, 75)
    logResult('TC-16-'+itemID, 'Bulk update for ' + itemID,
              'SUCCESS: Stock updated to 75', result)
END FOR
END FUNCTION

```

4. Main Test Runner

```

FUNCTION runAllTests():
    PRINT '===== INVENTORY UPDATE STOCK TEST SUITE ====='
    testItemAvailability()
    testQuantityLimits()
    testSystemResponse()

    // ■■■■ GENERATE SUMMARY REPORT ■■■■
    passed = COUNT results WHERE status == 'PASS' IN testResults
    failed = COUNT results WHERE status == 'FAIL' IN testResults
    total  = LENGTH(testResults)

    PRINT ''
    PRINT '===== TEST SUMMARY ====='
    PRINT 'Total   : ' + total
    PRINT 'Passed  : ' + passed
    PRINT 'Failed  : ' + failed

    IF failed > 0 THEN
        PRINT 'RESULT: SOME TESTS FAILED – Review defects.'
    ELSE
        PRINT 'RESULT: ALL TESTS PASSED.'
    END IF
END FUNCTION

// Entry point
CALL runAllTests()

```

5. Test Case Summary Table

TC ID	Description	Category	Expected Result
TC-01	Valid active item update	Availability	SUCCESS

TC-02	Non-existent item ID	Availability	ERROR: Not found
TC-03	Inactive item update	Availability	ERROR: Inactive
TC-04	Empty string item ID	Availability	ERROR: Not found
TC-05	NULL item ID	Availability	ERROR: Not found
TC-06	Quantity = 0 (min boundary)	Quantity Limit	SUCCESS
TC-07	Quantity = -1 (below min)	Quantity Limit	ERROR: Out of range
TC-08	Quantity = 10000 (max boundary)	Quantity Limit	SUCCESS
TC-09	Quantity = 10001 (above max)	Quantity Limit	ERROR: Out of range
TC-10	Quantity = 500 (mid-range)	Quantity Limit	SUCCESS
TC-11	Decimal quantity (10.5)	Quantity Limit	ERROR: Invalid type
TC-12	String quantity ('fifty')	Quantity Limit	ERROR: Invalid type
TC-13	Stock persists after update	System Response	Stock = 150
TC-14	Failed update — stock unchanged	System Response	Stock unchanged
TC-15	Sequential updates	System Response	Stock = last value
TC-16	Bulk/concurrent update	System Response	SUCCESS (all)

Conclusion

The pseudocode above provides a complete, executable test framework for the Update Stock module. By covering item availability, boundary values for quantity limits, and system response verification, the test suite ensures the module behaves correctly under valid, invalid, and edge-case conditions. This structured approach (BL3) demonstrates the ability to both **generate** and **execute** meaningful test cases aligned with real-world inventory system requirements.